

CIS 3300 • INTERMEDIATE OBJECT-ORIENTED PROGRAMMING

Classes, Objects, and Interfaces

From one class to a small working program

Bundling data with its behavior keeps the rules in one place.

WITHOUT ENCAPSULATION

- mileage is a public field
- any code, anywhere, can set it directly
- every caller has to remember the rule

```
car.mileage = -50;  
// compiles. wrong, and nothing stops it.
```

WITH ENCAPSULATION

- mileage is a private field
- setMileage() checks before changing it
- the rule lives in exactly one place

```
car.setMileage(-50);  
// rejected. the object protects itself.
```

Every object has three characteristics: state, behavior, and identity.



State

An object's properties and current values, its condition right now. Lives in an object's fields.



Behavior

The actions an object can perform, how it responds to interactions. Lives in an object's methods.



Identity

What makes one object different from another, even with identical state. Represented, in part, by a hash code.

A class bundles four kinds of pieces around one purpose.

```
class Car
```

Fields

Data every Car object carries: make, model, year, mileage.

Constructor

Runs once, when new Car(...) executes, to set up a valid starting object.

Methods

Public behavior: getters, setters, and actions such as drive(int miles).

Private helpers

Internal support methods, such as isValidMileage(), callable only from inside the class.

An access modifier controls how far a piece is visible.

`public`

Reaches everywhere: any class, in any package

`protected`

Reaches the same package, and subclasses in other packages

`default`

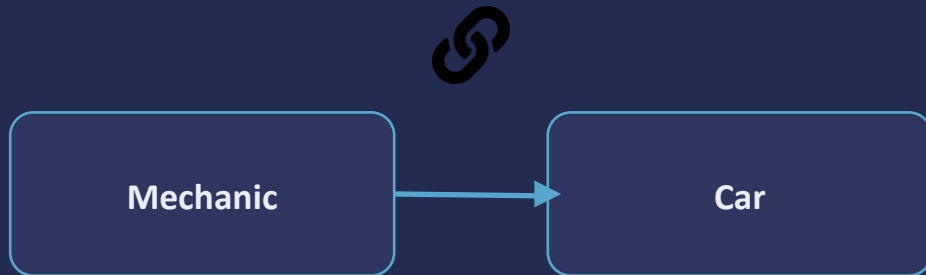
Reaches only other classes in the same package

`private`

Reaches only inside this one class

Two relationships come up constantly, short of full inheritance.

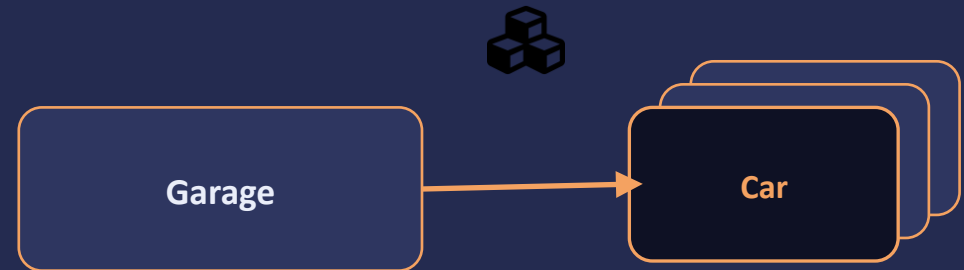
USES-A (dependence)



```
void inspect(Car car) { ... }
```

A Mechanic borrows a Car for one method call. It never stores one.

HAS-A (aggregation)



- Unidirectional: Garage knows Car; Car does not know Garage.
- Independent lifecycles: losing a Car does not break the Garage.

```
ArrayList<Car> fleet;
```

A Garage owns a list of Cars and manages them over time.

ONE MORE RELATIONSHIP IS COMING

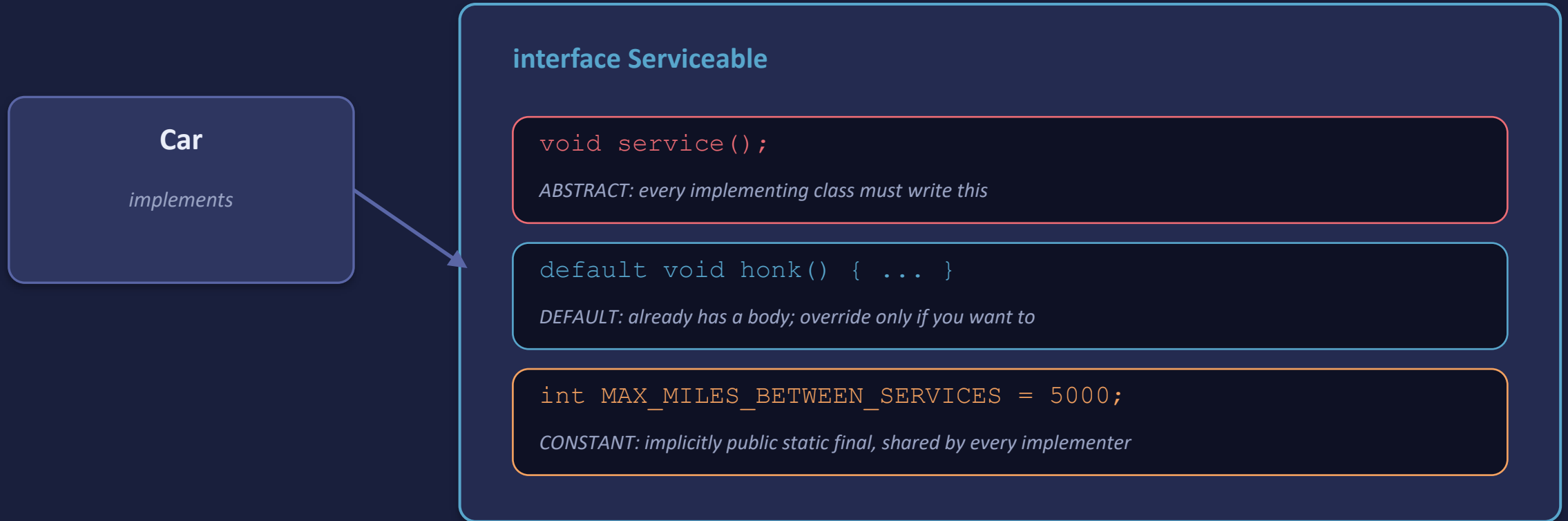
A SportsCar is-a Car: a specialized version that still works anywhere a Car does.



Unlike has-a and uses-a, is-a comes with a dedicated keyword: extends.

A preview only. Inheritance gets its own unit, later in the semester.

An interface is a contract: unrelated classes can promise the same behavior.



From slides to a running program, one class at a time.

- 1 The anatomy of a class, built live, then instantiated and used.
- 2 Private helper methods, and a second class we build together.
- 3 Passing objects to methods, and storing objects in an ArrayList.
- 4 Overriding toString, equals, and hashCode on our own classes.
- 5 Interfaces and default methods, in code this time.

Inheritance (is-a) waits for its own unit, later in the semester.